5. Gyakorlat

Algoritmusok és adatszerkezetek 2.

Törlés B+ fákban

B+ fa - emlékeztető

- Levélben: minimum floor(d/2), maximum d-1 kulcs és ugyanennyi mutató, ami az adatrekordra mutat
- Minden levél azonos mélységben, a legalsó szinten van
- Belső csúcsokban: eggyel több mutató, mint hasítókulcs.
  - A hasítókulcsok közötti részfákban a hasítókulcsok közé eső kulcsok esnek.
- Belső csúcsoknak minimum floor(d/2) gyereke van, maximum d
  - Kulcsokból eggyel kevesebb, tehát minimum floor(d/2)-1, maximum d-1
- A gyökércsúcsnak legalább két gyereke van (kivéve, ha ez a fa egyetlen csúcsa, következésképpen az egyetlen levele is).
- A B+ fa által reprezentált adathalmaz minden kulcsa megjelenik valamelyik levélben, balról jobbra szigorúan monoton növekvő sorrendben

Törlés - 4 eset

A) Levélcsúcs = gyökér

B) Levélcsúcs nem a gyökért

C) Belsős csúcsból kell törölni - mert felfele kellett lépnünk

D) A gyökérig fel kell menni

Törlés

A, A keresés során megtalált levélcsúcs egyben a gyökércsúcs is:

1. Töröljük a megfelelő kulcsot és a hozzá tartozó mutatót a csúcsból! 2. Ha a csúcs tartalmaz még kulcsot, kész vagyunk.

3. Különben töröljük a fa egyetlen csúcsát, és üres fát kapunk.

Törlés - B

B, A keresés során megtalált levélcsúcs nem a gyökércsúcs:

1. Töröljük a megfelelő kulcsot és a hozzá tartozó mutatót a levélcsúcsból!

2. Ha a levélcsúcs még tartalmaz elég kulcsot és mutatót, hogy teljesítse az invariánsokat, kész vagyunk.

3. Ha a levélcsúcsban már túl kevés kulcs van ahhoz, hogy teljesítse az invariánsokat, de a következő, vagy a megelőző testvérének több van, mint amennyi szükséges, osszuk el a kulcsokat egyenlően közte és a megfelelő testvére között! Írjuk át a két testvér közös szülőjében a két testvérhez tartozó hasító kulcsot a két testvér közül a második minimumára!

4. Ha a levélcsúcsban már túl kevés kulcs van ahhoz, hogy teljesítse az invariánst, és a következő, valamint a megelőző testvére is a minimumon van, hogy teljesítse az invariánst, akkor egyesítsük egy vele szomszédos testvérével! Ennek során a két testvér közül a (balról jobbra sorrend szerinti) másodikból a kulcsokat és a hozzájuk tartozó mutatókat sorban átmásoljuk az elsőbe, annak eredeti kulcsai és mutatói után, majd a második testvért töröljük. Ezután meg kell ismételnünk a törlő algoritmust a szülőre, hogy eltávolítsuk a szülőből a hasító kulcsot (ami eddig elválasztotta a most egyesített levélcsúcsokat), a most törölt második testvérre hivatkozó mutatóval együtt.

Törlés C

1. Töröljük a belső csúcs éppen most egyesített két gyereke közti hasító kulcsot és az egyesítés során törölt gyerekére hivatkozó mutatót a belső csúcsból!
2. Ha a belső csúcsnak van még floor(d/2) gyereke, (hogy teljesítse az invariánsokat) kész vagyunk.
3. Ha a belső csúcsnak már túl kevés gyereke van ahhoz, hogy teljesítse az invariánsokat, de a következő, vagy a megelőző testvérének több van, mint amennyi szükséges, osszuk el a gyerekeket és a köztük levő hasító kulcsokat egyenlően közte és a megfelelő testvére között, a hasító kulcsok közé a testvérek közti (a közös szülőjükben lévő) hasító kulcsot is beleértve! A gyerekek és a hasító kulcsok újraelosztása során, a középső hasító kulcs a testvérek közös szülőjében a két testvérhez tartozó régi hasító kulcs helyére kerül úgy, hogy megfelelően reprezentálja a köztük megváltozott vágási pontot! (Ha a két testvérben a gyerekek összlétszáma páratlan, akkor az újraelosztás után is annak a testvérnek legyen több gyereke, akinek előtte is több volt!)
4. Ha a belső csúcsnak már túl kevés gyereke van ahhoz, hogy teljesítse az invariánst, és a következő, valamint a megelőző testvére is a minimumon van, hogy teljesítse az invariánst, akkor egyesítsük egy vele szomszédos testvérével! Az egyesített csúcsot a két testvér közül a (balról jobbra sorrend szerinti) elsőből hozzuk létre. Gyerekei és hasító kulcsai először a saját gyerekei és hasító kulcsai az eredeti sorrendben, amiket a két testvér közti (a közös szülőjükben lévő) hasító kulcs követ, és végül a második testvér gyerekei és hasító kulcsai jönnek, szintén az eredeti sorrendben. Ezután töröljük a második testvért. A két testvér egyesítése után meg kell ismételnünk a törlő algoritmust a közös szülőjükre, hogy eltávolítsuk a szülőből a hasító kulcsot (ami eddig elválasztotta a most egyesített testvéreket), a most törölt második testvérre hivatkozó mutatóval együtt.

Törlés D

1. Töröljük a gyökércsúcs éppen most egyesített két gyereke közti hasító kulcsot és az egyesítés során törölt gyerekére hivatkozó mutatót a gyökércsúcsból!

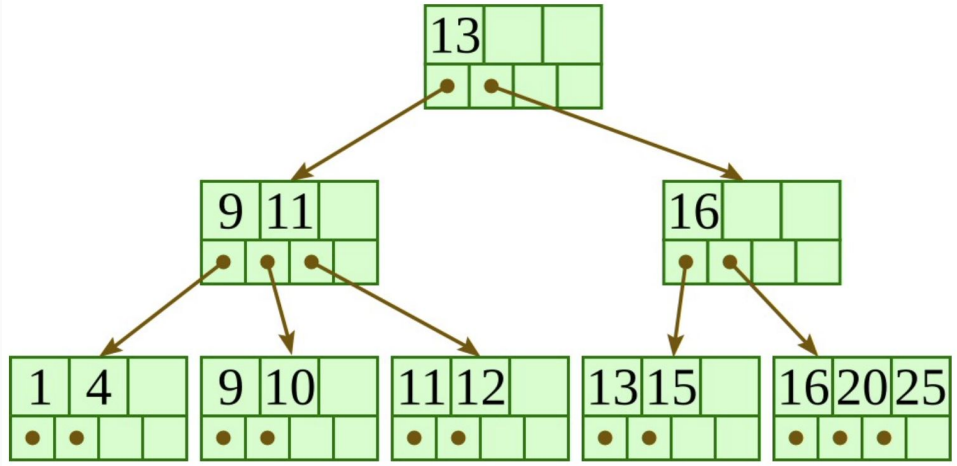
2. Ha a gyökércsúcsnak van még 2 gyereke, kész vagyunk.

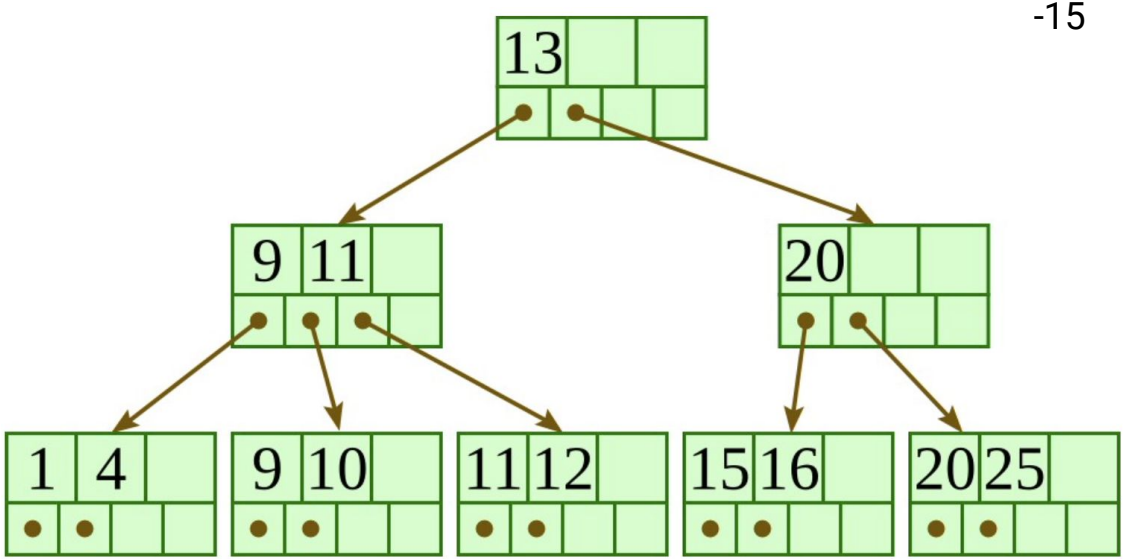
3. Ha a gyökércsúcsnak csak 1 gyereke maradt, akkor töröljük a gyökércsúcsot, és a megmaradt egyetlen gyereke legyen az új gyökércsúcs! (Ekkor a B+ fa magassága csökken.)

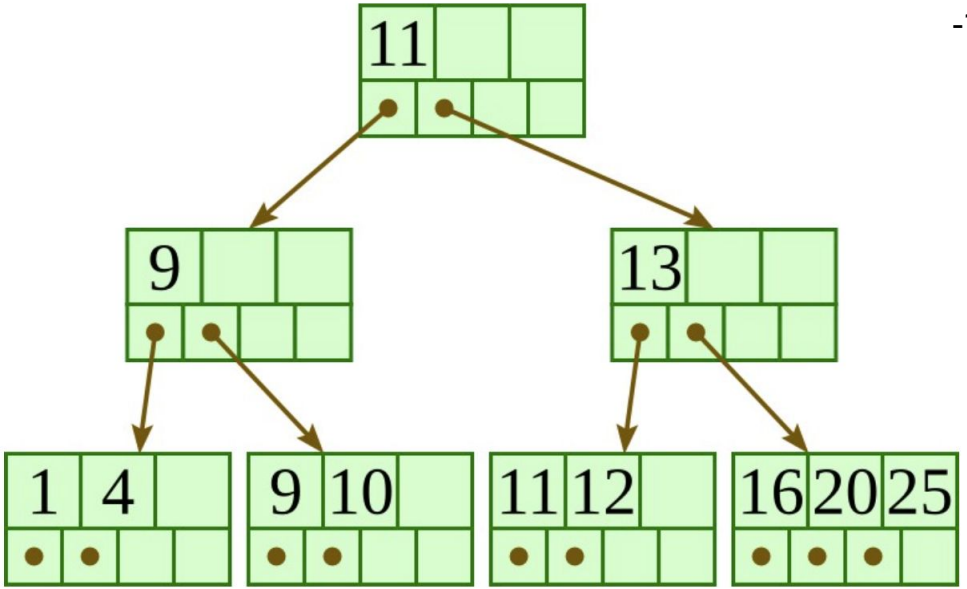
5. Feladat

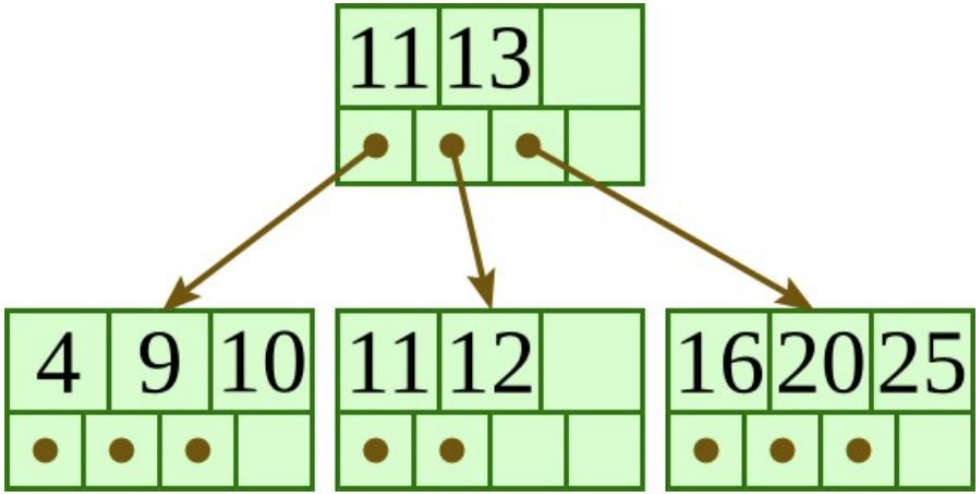
Töröljük az alábbi fából sorban a következő elemeket:

13,15,1




13
9 11
20
1 4
9 10
11 12
15 16
20 25


11
9
13
1
4
9
10
11
12
16
20
25


11 13
4 9 10
11 12
16 20 25

Gráfok

Definíció

Gráf alatt egy G = (V, E) rendezett párost értünk, ahol V a csúcsok vagy pontok (vertices) tetszőleges, véges halmaza, E ’részhalmaza’ V x V \ {(u, u) : u V} pedig az élek (edges) halmaza. Az élek E halmazának elemei bizonyos V-beli párok. Ha V = { }, akkor üres gráfról, ha V { }, akkor nemüres gráfról beszélünk.

Általános jelölés:

n = |V| - csúcsok száma

e = |E| - élek száma

Milyenek lehetnek a gráfok?

Irányított - irányítatlan

Súlyozott - súlyozatlan

- Megengedünk negatív élcsúcsot vagy sem?
- Súlyfüggvény

Többszörös élek, hurokélek

Fogalmak

Út: $v_1, .. v_k$ csúcsok sorozata, úgy, hogy $i \in [1..k-1]: (v_i, v_{i+1}) \in E$

Kör: olyan út, hogy $v_k = v_1$

Ritka gráf: $|E| << |V|^2$

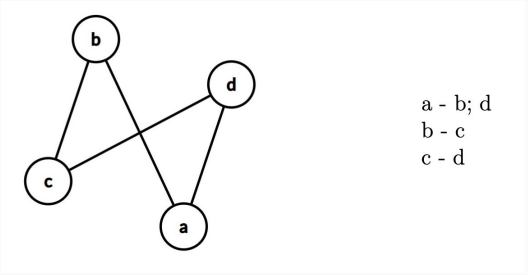
Sűrű gráf: $|E| \approx |V|^2$

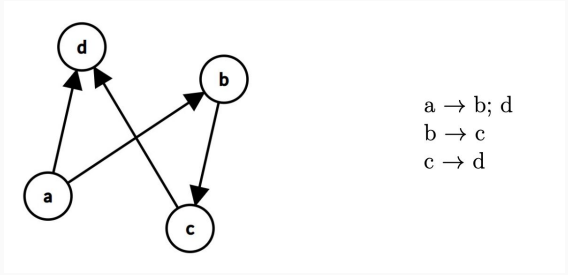
Gráfok szemléltetése

Grafikus - lerajzoljuk a csúcsokat és az összekötéseket

Szöveges - leírjuk, hogy honnan hova vezet él

Irányított, irányítatlan?





Pár kérdés, amit feltehetünk

- Hogyan jutunk el egy csúcsból egy másikba?
- Van-e kör egy gráfban?
- Mi a legolcsóbb út két csúcs között?
- Milyen éleket hagyhatunk el, hogy összefüggő gráfot kapjunk? Melyik a “legolcsóbb” ilyen gráf.
- Hatékony algoritmusok léteznek ezek eldöntésére.
- Számtalan felhasználási terület a való életben

Gráfok ábrázolása

Éllistás ábrázolás // adjacency list

Az n csúcsú gráf ekkor pointerek tömbje, amelyet a csúcsokkal indexelünk - A[1..n]

Minden elem típusa E1* és S1L (egyszerű, egy irányú) listaként kezeljük az éllistát.

Implementáció esetén használhatunk nagyobb tudású adattípust

Lista típus, beszúrási, keresési, stb. művelettel

Súlyozott gráf esetén az élsúlyt felvesszük a listaelembe

Szomszédsági mátrix // adjacency matrix

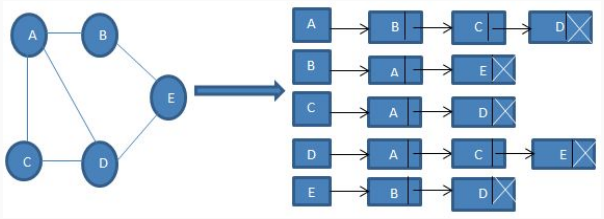
Az n csúcsú gráf ilyenkor egy A csúcsmátrix, amelynek mérete n*n

Ha (i,j) él eleme a gráfnak, akkor A[i,j] = 1, különben 0

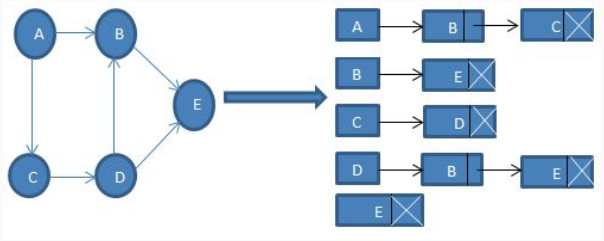
Élsúlyok: A[i,j] = élsúly

Éllistás és szomszédsági mátrix példák

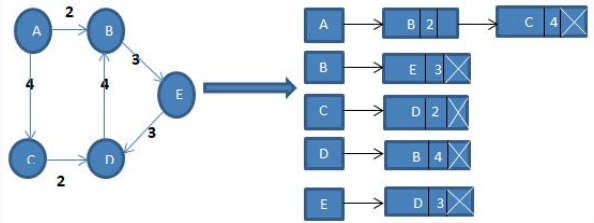
irányítatlan:



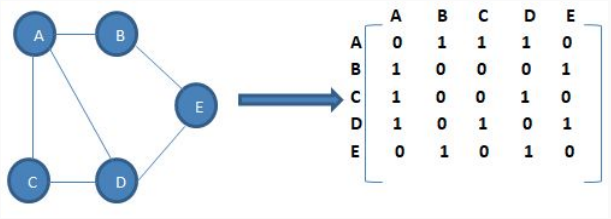
irányított:



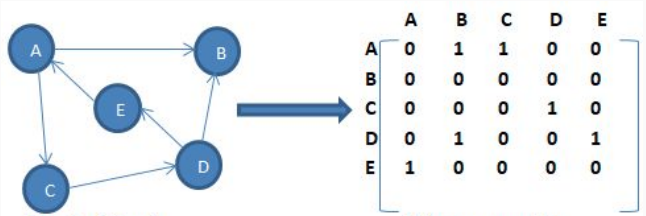
élsúlyozott:



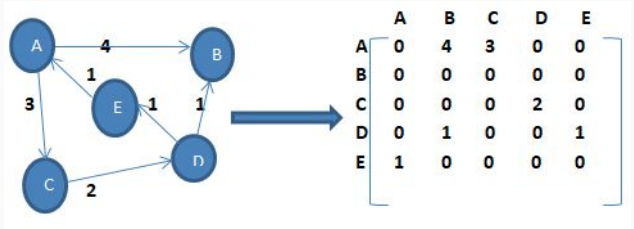
irányítatlan:



irányított:

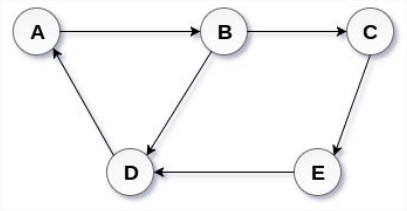


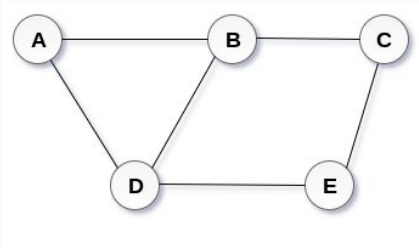
élsúlyozott:

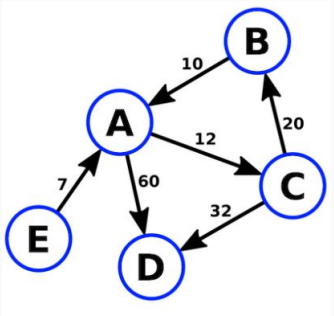


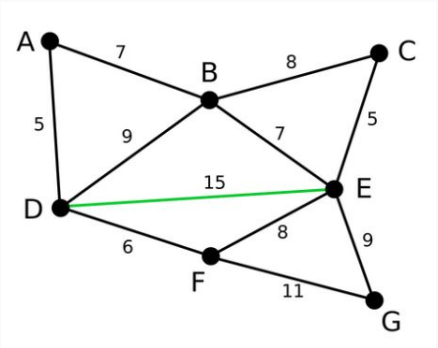
0. Feladat

Adjuk meg az alábbi gráfok ábrázolásait mindkét ábrázolási formában:









1. Feladat

Adott egy irányított G gráf, amelynek n db. csúcsa van éllistás ábrázolással. Ezt egy T[1..n] tömbként érjük el. Transzformáljuk át csúcsmátrixos ábrázolásba.

2. Feladat

Adott egy G irányítatlan gráf, amelynek n db. csúcsa van szomszédsági mátrixos ábrázolással. A szomszédsági mátrixnak csak a felső háromszöge van kitöltve. Transzformáljuk át éllistás ábrázolásba.

3. Feladat

Adott egy irányított egyszerű G gráf, amelynek n db. csúcsa van éllistás ábrázolással megadva. Állítsunk elő egy G’ gráfban a G gráf fordítottját.

4. Feladat

Adott egy táblázat, amelyben egy város metróhálózata adott a jobbra látható formában.

Ha két vonalon szerepel ugyanaz a megálló, annak jelentése, hogy át lehet szállni a két vonal között.

Tervezzünk ábrázolást arra az esetre, ha a megállók számának figyelembe vételével szeretnénk a későbbiekben algoritmust írni és tervezzünk olyan ábrázolást is, amelyben az átszállásokra szeretnénk optimalizálni a későbbiekben.

Vonal 1: Megálló 1, Megálló 2, … Megálló i_1

Vonal 2: Megálló 1, Megálló 2, … Megálló i_2

…

Vonal n: Megálló 1, Megálló 2, … Megálló i_n